

1. Constantes

¿De qué manera se define una constante?

En PHP, una constante se puede definir de dos maneras:

1. Usando la función `define()`:

- `define('NOMBRE_CONSTANTE', 'valor');`
- Esto define una constante en tiempo de ejecución. Es útil para definir constantes basadas en una condición.
- Por convención, los nombres de las constantes se escriben en mayúsculas.
- Ejemplo: `define('VERSION', '1.0.2');`

2. Usando la palabra clave `const`:

- `const NOMBRE_CONSTANTE = 'valor';`
- Esto define una constante en tiempo de compilación. Es más rápido que `define()`.
- A partir de PHP 5.3, `const` también se puede usar fuera de las clases.
- Ejemplo: `const PI = 3.1416;`

¿En qué ocasiones es necesario realizar una declaración de una constante?

Se usan constantes para almacenar valores que **no deben cambiar** durante la ejecución del script. Son útiles para:

- **Configuración:** Credenciales de base de datos (usuario, contraseña, host), claves de API, rutas de directorios.
- **Valores Mágicos:** Valores fijos que tienen un significado especial, como un estado (`define('ESTADO_PUBLICADO', 1);`).
- **Legibilidad:** Dar un nombre descriptivo a un valor fijo (como `const IVA = 0.16;`) en lugar de usar el número directamente en el código.

¿Cuáles son las constantes que están predefinidas?

PHP tiene muchas constantes predefinidas. Las más comunes son las "Constantes Mágicas", que cambian su valor dependiendo de dónde se usen:

- `__LINE__`: El número de la línea actual en el archivo.
- `__FILE__`: La ruta completa y el nombre del archivo.
- `__DIR__`: El directorio del archivo.
- `__FUNCTION__`: El nombre de la función.
- `__CLASS__`: El nombre de la clase.
- `__METHOD__`: El nombre del método de la clase.
- `__NAMESPACE__`: El nombre del espacio de nombres actual.

También existen otras constantes predefinidas como `PHP_VERSION` (versión de PHP), `PHP_OS` (sistema operativo), y constantes de reporte de errores como `E_ERROR`, `E_WARNING`, etc.

2. Variables de Entorno

¿Qué es una variable de entorno?

Una variable de entorno es una variable cuyo valor se establece **fuera** del programa (en el sistema operativo o en el servidor web) y está disponible para el programa cuando se ejecuta.

¿Cómo se aplican Y de qué manera se utilizan en un entorno de desarrollo y de

producción?

Su propósito principal es **separar la configuración del código**. Esto es crucial para la seguridad y la portabilidad.

- **Aplicación:** En PHP, se acceden comúnmente usando `getenv('NOMBRE_VARIABLE')` o a través de la superglobal `$_ENV['NOMBRE_VARIABLE']`.
- **Entorno de Desarrollo (Development):**
 - **Uso:** Se suelen usar para configuraciones locales. Por ejemplo, la base de datos se llamará `mi_app_dev`.
 - **Método:** Es común usar un archivo `.env` (leído por una librería como `vlucas/phpdotenv`) que *no* se sube al repositorio de código (se ignora en `.gitignore`).
 - Ejemplo en `.env`:

```
DB_HOST=localhost
DB_USER=root
DB_PASS=root
APP_DEBUG=true
```
- **Entorno de Producción (Production):**
 - **Uso:** Almacenan configuraciones sensibles y específicas del servidor real, como la contraseña de la base de datos de producción o claves de API reales.
 - **Método:** Estas variables se configuran directamente en el servidor (por ejemplo, en la configuración de Apache/Nginx, en el panel de control de un hosting como Heroku, o en los secretos de AWS/Google Cloud).
 - Ejemplo en el servidor:

```
DB_HOST=rds.amazonaws.com
DB_USER=admin_prod
DB_PASS=S3cr3tP@ssw0rd
APP_DEBUG=false
```
 - El código PHP simplemente lee `getenv('DB_PASS')` y funcionará en ambos entornos sin necesidad de cambiar el código, ya que tomará el valor correspondiente al entorno en el que se está ejecutando.

3. Operadores Aritméticos y Lógicos

¿Cuáles son los operadores aritméticos?

- + : Adición
- - : Sustracción
- * : Multiplicación
- / : División
- % : Módulo (resto de la división)
- ** : Exponenciación (elevado a la potencia)

¿Cuáles son los operadores lógicos?

- and o &&: Y (verdadero si ambos son verdaderos)
- or o ||: O (verdadero si al menos uno es verdadero)
- xor: O Exclusivo (verdadero si solo uno es verdadero, pero no ambos)
- ! : No (invierte el valor de verdad)

4. Variables Súper Globales

Las variables súper globales son variables predefinidas en PHP que están siempre disponibles en todos los ámbitos (dentro de funciones, clases, etc.) sin necesidad de usar global.

- **\$_SERVER:** Contiene información sobre el servidor y el entorno de ejecución. Es un array enorme con datos como:
 - `$_SERVER['HTTP_HOST']`: El host de la solicitud (ej: `www.midominio.com`).
 - `$_SERVER['REQUEST_URI']`: La URI de la página (ej: `/blog/articulo.php?id=1`).
 - `$_SERVER['SCRIPT_NAME']`: La ruta del script actual (ej: `/blog/articulo.php`).
 - `$_SERVER['REMOTE_ADDR']`: La dirección IP del cliente.
 - `$_SERVER['REQUEST_METHOD']`: El método de la solicitud (ej: GET, POST).
- **\$_GET:** Un array asociativo de variables pasadas al script a través de la URL (query string).
 - Para la URL `index.php?usuario=juan&id=123`, `$_GET` sería `['usuario' => 'juan', 'id' => '123']`.
- **\$_POST:** Un array asociativo de variables pasadas al script a través del método HTTP POST (típicamente desde un formulario).
 - Si un formulario con `<input name="email">` se envía, el valor estará en `$_POST['email']`.
- **\$_FILES:** Un array asociativo con información sobre archivos subidos al servidor mediante un formulario (`enctype="multipart/form-data"`). Contiene el nombre del archivo, tipo, tamaño, nombre temporal y código de error.
- **\$_COOKIE:** Un array asociativo con los valores de las cookies HTTP enviadas por el navegador del cliente al servidor.
- **\$_SESSION:** Un array asociativo que almacena datos de sesión en el servidor. A diferencia de las cookies, la información no se almacena en el cliente (solo se guarda un ID de sesión en una cookie), lo que lo hace más seguro para datos sensibles (como el ID de un usuario logueado). Requiere `session_start()` para ser usada.
- **\$_REQUEST:** Un array asociativo que, por defecto, contiene una mezcla de

\$_GET,

\$_POST y \$_COOKIE. No se recomienda su uso por razones de seguridad, ya que no se sabe de dónde proviene el dato. Es mejor ser explícito y usar \$_GET o \$_POST.

5. Funciones y Programación Orientada a Objetos (POO)

¿En qué se diferencia una función de una clase?

- Una **Función** es un bloque de código reutilizable que realiza una *tarea* o *acción* específica (ej: sumarNumeros(), conectarDB()).
- Una **Clase** es una *plantilla* o *molde* para crear objetos. Define un *concepto* o *entidad* (ej: Usuario, Producto), especificando qué *propiedades* (datos) tiene y qué *métodos* (funciones/comportamiento) puede realizar.

¿Qué es una clase?

Es la plantilla. Define la estructura y el comportamiento de un tipo de objeto. Ejemplo:

```
class Coche {  
    // Propiedad  
    public $color = 'rojo';  
  
    // Método (función dentro de una clase)
```

```

        public function arrancar() {
            echo 'El coche ha arrancado.';
        }
    }
}

```

¿Qué es un objeto?

Es una **instancia** de una clase. Es la "cosa" real creada a partir de la plantilla.

```

// $miCoche es un objeto (una instancia de la clase Coche)
$miCoche = new Coche();
echo $miCoche->color; // Accede a la propiedad
$miCoche->arrancar(); // Llama al método

```

¿Qué es la herencia?

Es un mecanismo de la POO que permite a una clase (clase hija) heredar propiedades y métodos de otra clase (clase padre). Se usa la palabra clave `extends`.

```

class CocheDeportivo extends Coche {
    public function activarTurbo() {
        echo 'Turbo activado.';
    }
}

```

```

$miDeportivo = new CocheDeportivo();
$miDeportivo->arrancar(); // Método heredado de Coche
$miDeportivo->activarTurbo(); // Método propio

```

¿Qué es el polimorfismo?

Significa "muchas formas". Es la capacidad de los objetos de diferentes clases de responder al mismo mensaje (llamada de método) de maneras diferentes. Se logra comúnmente mediante la *sobrescritura de métodos* (la clase hija redefine un método de la clase padre).

¿Qué es el POO y sus características?

POO (Programación Orientada a Objetos) es un paradigma de programación basado en el concepto de "objetos". Sus características (pilares) principales son:

1. **Encapsulamiento:** Ocultar el estado interno (propiedades) de un objeto y

exponer solo lo necesario a través de métodos (usando public, protected, private).

2. **Abstracción:** Ocultar la complejidad interna y mostrar solo las características esenciales de un objeto (similar al encapsulamiento, pero más conceptual).
3. **Herencia:** (Definida arriba). Permite la reutilización de código.
4. **Polimorfismo:** (Definido arriba). Permite flexibilidad y código más genérico.

6. Estructura MVC

¿Qué es?

MVC (Modelo-Vista-Controlador) es un patrón de arquitectura de software que separa una aplicación en tres componentes principales:

1. **Modelo (Model):** Representa los datos y la lógica de negocio. Es responsable de interactuar con la base de datos, realizar cálculos y aplicar reglas de negocio. (Ej: una clase Usuario que sabe cómo buscarse en la BBDD).
2. **Vista (View):** Es la capa de presentación, lo que el usuario ve (el HTML, CSS). Su único

trabajo es mostrar los datos que le pasa el Controlador. No debe contener lógica de negocio.

3. **Controlador (Controller):** Actúa como el intermediario. Recibe la solicitud del usuario (HTTP request), le pide al Modelo los datos necesarios, y luego le pasa esos datos a la Vista para que los muestre.

¿Cómo aplicarlo en el desarrollo con PHP?

En PHP "puro" (sin un framework), se puede implementar de forma básica así:

1. **Estructura de archivos:**

```
/mi_proyecto
  /models
    Usuario.php
  /views
    inicio.php
    perfil.php
  /controllers
    UsuarioController.php
  index.php
```

2. **index.php (Front Controller):** Todas las solicitudes entran por aquí. Este archivo lee la URL (ej: index.php?accion=verPerfil&id=5).
3. **Router (en index.php):** Decide qué Controlador y método llamar.
 - Si accion=verPerfil, llama a UsuarioController->verPerfil(5).

4. **UsuarioController.php:**

```
class UsuarioController {
    public function verPerfil($id) {
        // 1. Pide datos al Modelo
        require_once
            '../models/Usuario.php';
        $usuario = new Usuario();
        $datosUsuario = $usuario->obtenerPorId($id);

        // 2. Pasa los datos a la Vista
        require '../views/perfil.php';
    }
}
```

5. **Usuario.php (Modelo):**

```
class Usuario {  
    public function obtenerPorId($id) {  
        // ...lógica para conectar a la BBDD y hacer "SELECT  
* FROM usuarios WHERE id = $id"  
        return $datosDeLaBDD;  
    }  
}
```

6. **perfil.php (Vista):**

```
<h1>Perfil de <?php echo $datosUsuario['nombre']; ?></h1>  
<p>Email: <?php echo $datosUsuario['email']; ?></p>
```

7. Funciones Predefinidas de PHP

- **Debugging:** `var_dump($variable)` (muestra tipo y valor), `print_r($variable)` (muestra el contenido de un array/objeto de forma legible).
- **Verificar Fecha/Tiempo/Ambos:** `checkdate($mes, $dia, $anio)` (verifica si una fecha gregoriana es válida), `strtotime($string)` (convierte una fecha/hora en texto a un timestamp Unix), `DateTime::createFromFormat($formato, $string)` (crea un objeto de fecha a partir de un formato específico).
- **Raíz cuadrada:** `sqrt($numero)`
- **Número aleatorio:** `rand($min, $max)` (simple), `mt_rand($min, $max)` (mejor, más rápido), `random_int($min, $max)` (criptográficamente seguro, para tokens, contraseñas, etc.).
- **Número Pi:** `pi()`
- **Redondear:** `round($numero)` (al más cercano), `ceil($numero)` (redondea hacia arriba), `floor($numero)` (redondea hacia abajo).
- **Obtener tipo de variable:** `gettype($variable)` (devuelve un string como "integer", "string", "array").
- **Verificar si es...:** `is_string()`, `is_float()` (o `is_double()`), `is_int()`, `is_array()`, `is_bool()`, `is_numeric()`.
- **Verificar si existe:** `isset($variable)` (devuelve true si la variable está definida y no es null).
- **trim(\$string):** Elimina espacios en blanco (u otros caracteres especificados) del inicio y el final de un string.
- **unset(\$variable):** Destruye (elimina) una variable.
- **empty(\$variable):** Comprueba si una variable está "vacía". Devuelve true para "" (string vacío), 0, "0", null, false, y un array() vacío.
- **strlen(\$string):** Devuelve la longitud (número de bytes) de un string.
- **strpos(\$string, \$sub_string):** Encuentra la posición de la primera ocurrencia de un sub-string.
- **str_replace(\$buscar, \$reemplazar, \$string):** Reemplaza todas las ocurrencias de \$buscar con \$reemplazar en \$string.
- **strtolower(\$string):** Convierte un string a minúsculas.
- **strtoupper(\$string):** Convierte un string a mayúsculas.

8. Sanitización de Datos con Funciones

¿De qué manera se aplican esas funciones para sanitizar datos antes del envío de un formulario?

La **sanitización** es el proceso de "limpiar" los datos de entrada para eliminar caracteres no deseados o potencialmente peligrosos. Es diferente de la **validación** (que comprueba si los datos son correctos).

Antes de *guardar* en la base de datos o *usar* los datos de un formulario (\$_POST), se deben sanitizar:

1. Eliminar espacios extra:

```
$nombre = $_POST['nombre'];  
$nombre_limpio = trim($nombre);
```

2. Estandarizar (ej. email):

```
$email = $_POST['email'];  
$email_limpio = strtolower(trim($email));
```

3. Eliminar etiquetas HTML (para evitar XSS si se va a mostrar):

```
$comentario = $_POST['comentario'];  
// La forma moderna y correcta de sanitizar para mostrar en  
HTML:  
$comentario_seguro_para_html = htmlspecialchars($comentario,  
ENT_QUOTES, 'UTF-8');  
  
// Una forma más "agresiva" (elimina las etiquetas en lugar  
de escaparlas):  
$comentario_limpio = strip_tags($comentario);
```

4. Función recomendada (filtros): La mejor manera de sanitizar en PHP es usar filter_var().

```
$email = $_POST['email'];  
$email_sanitizado = filter_var($email,  
FILTER_SANITIZE_EMAIL);  
  
$numero = $_POST['edad'];  
$numero_sanitizado = filter_var($numero,  
FILTER_SANITIZE_NUMBER_INT);
```

9. Seguridad: XSS y DoS

¿Cómo evitar un Cross-site Scripting (XSS)?

El XSS ocurre cuando un atacante inyecta código malicioso (usualmente JavaScript) en tu página web, que luego se ejecuta en el navegador de otra víctima.

Prevención: La regla de oro es: **NUNCA confíes en los datos del usuario.**

- **Al mostrar datos en HTML:** Usa htmlspecialchars(). Esta función convierte caracteres especiales de HTML en sus entidades, volviéndolos inofensivos.
 - < se convierte en <
 - > se convierte en >
 - " se convierte en "

0 **Ejemplo:**

```
// El usuario envía: <script>alert('hackeado');</script>
$comentario = $_POST['comentario'];
```

```
// MODO INSEGURO:
```

```
echo '<div>' . $comentario . '</div>'; // Ejecutará el
script
```

```
// MODO SEGURO:
```

```
echo '<div>' . htmlspecialchars($comentario,
ENT_QUOTES, 'UTF-8') . '</div>';
```

```
// El navegador mostrará el texto literal
```

```
"<script>alert('hackeado');</script>"
```

¿Cómo evitar un DoS (Denial of Service)?

Un DoS (o DDoS, si es Distribuido) es un ataque que busca sobrecargar tu servidor con tantas solicitudes que no pueda responder a los usuarios legítimos.

Prevención (a nivel de PHP/Aplicación):

- **Rate Limiting (Limitación de Tasa):** No permitir que una misma IP haga demasiadas solicitudes por minuto (ej: en un login o formulario de contacto).
- **Caching (Caché):** Guardar copias de páginas pesadas (como la portada) para no tener que generarlas (consultando la BBDD) en cada visita.
- **Validación y Captcha:** Usar un CAPTCHA en formularios para asegurar que es un humano y no un bot.

(Nota: La mayoría de la protección contra DDoS se hace a nivel de red y servidor (ej: Cloudflare, firewalls), no solo en el código PHP).

¿Cuáles son sus afectaciones en el sistema y base de datos?

- **XSS:**
 - **Sistema:** No afecta directamente al servidor.
 - **Base de Datos:** No la corrompe, pero si el XSS se almacena en la BBDD (XSS Persistente), infectará a todos los usuarios que vean ese dato (ej: un comentario malicioso).
 - **Afectación principal:** Al *cliente* (usuario). Le pueden robar sus cookies de sesión, credenciales, o redirigirlo a sitios de phishing.
- **DoS:**
 - **Sistema:** Lo *tumba*. Satura la CPU, la memoria, el ancho de banda y las conexiones de red, haciendo que el sitio sea inaccesible.
 - **Base de Datos:** Puede saturar la base de datos con consultas (si el ataque se enfoca ahí), agotando su límite de conexiones y volviéndola también inaccesible.

10. Inclusión de Archivos (include, require)

- **include 'archivo.php';**
 - Intenta incluir y evaluar el archivo.
 - Si el archivo **no existe**, genera un **Warning** (advertencia) y el script **continúa** ejecutándose.
 - Útil para plantillas opcionales (ej: un widget).
- **require 'archivo.php';**

- Requiere el archivo para funcionar.
- Si el archivo **no existe**, genera un **Fatal Error** (error fatal) y el script **se detiene** por completo.
- Útil para archivos críticos (ej: conexión a BBDD, funciones principales).
- **include_once 'archivo.php';**
 - Igual que include, pero PHP verifica si el archivo ya fue incluido antes. Si es así, **no lo vuelve a incluir**.
- **require_once 'archivo.php';**
 - Igual que require, pero PHP verifica si el archivo ya fue incluido antes. Si es así, **no lo vuelve a incluir**.

¿Por qué es importante incluirlos?

- **Organización:** Permiten seguir el principio **DRY (Don't Repeat Yourself - No te repitas)**.
- **Reutilización:** Puedes tener un solo archivo header.php o database.php y usarlo en 20 páginas diferentes.

- **Mantenimiento:** Si necesitas cambiar el menú de navegación, solo editas header.php en lugar de 20 archivos.
- `_once` es vital al trabajar con **clases y funciones** para evitar errores fatales de "redeclaración" (Cannot redeclare function...).

11. Arrays

¿Qué es un array normal (indexado)?

Un array donde las claves (índices) son numéricos y automáticos (empiezan en 0).

```
$frutas = ['Manzana', 'Pera', 'Naranja'];
// $frutas[0] es 'Manzana'
// $frutas[1] es 'Pera'
```

¿Qué es un array asociativo?

Un array donde las claves (índices) son *strings* personalizados (asocias un nombre a un valor).

```
$usuario = [
    'nombre' =>
    'Carlos', 'edad' =>
    30, 'ciudad' =>
    'Madrid'
];
// $usuario['nombre'] es 'Carlos'
```

¿Qué es un array multidimensional?

Un array que contiene otros arrays.

```
$usuarios = [
    [
        'nombre' => 'Carlos',
        'edad' => 30
    ],
    [
        'nombre' => 'Ana',
        'edad' => 25
    ]
]
```

```
];  
// $usuarios[1]['nombre'] es 'Ana'
```

¿Cuáles son los diferentes tipos de funciones para array que existen en PHP?

Hay muchísimas. Se pueden agrupar por su propósito:

- **Creación y Conteo:** array(), count(), sizeof().
- **Añadir/Quitar elementos:** array_push() (añade al final), array_pop() (quita del final), array_unshift() (añade al inicio), array_shift() (quita del inicio).
- **Búsqueda:** in_array() (comprueba si un valor existe), array_search() (busca un valor y devuelve su clave).
- **Ordenamiento:** sort() (ordena valores), rsort() (ordena al revés), asort() (ordena valores manteniendo claves), ksort() (ordena por clave).
- **Manipulación:** array_keys() (devuelve las claves), array_values() (devuelve los valores), array_merge() (fusiona arrays), array_slice() (corta una porción).
- **Iteración (Funcionales):** array_map() (aplica una función a cada elemento), array_filter()

(filtra elementos según una función).